

HARP: released code vs. published paper

The setup mismatch, and what it is worth in AUROC

August 7, 2026

Purpose. We reran HARP’s released code on our own generated data so that our comparison against their published numbers is like-for-like. While reading their code to build the data adapter we found one place where the released implementation does not match the setup the paper describes. This document reports the audit, the reproduction, and a controlled experiment that measures what the mismatch is worth.

Summary. The *method* is implemented exactly as the paper describes – detector architecture, token aggregation, optimiser, and correctness thresholds all match. The one substantive difference is **how the data is split into training and test sets**. On our data that difference is worth **3.8 to 13.3 AUROC points** to HARP’s detector – and **1.3 to 11.8 points to ours**. It is a property of the protocol, not of their method: any evaluation that splits multi-sample generations over answers rather than questions inherits it, including ours when we run it that way.

1. The audit

2. The mismatch in plain terms

What is *not* wrong. Questions are correctly labelled known or unknown per the paper’s definition; all unknown questions are correctly placed in the test set and never used for training; the 75/25 ratio is correctly applied only to the known group. There is a genuine known/unknown separation.

What is wrong is one level further down. Each question is answered ten times. The paper splits the known group *over questions*; the code pools that group’s answers into one list and splits *that* at random. A known question therefore keeps all ten answers on the training side only about 5.6% of the time (0.75^{10}) and almost never lands entirely in the test set, so roughly **94% of known questions contribute answers to both sides**. For those, the question is not unseen at test time.

This is directly measurable, and the run below confirms it: the number of known questions appearing on both sides of the split was 626 of 666 (TruthfulQA), 5,956 of 6,316 (TriviaQA), 334 of 360 (NQ-Open) and 278 of 302 (TyDiQA-GP) – **92–94%**, against the 94.4% predicted.

3. The reproduction

We fed our own pinned generations into their generation cache (`main.py:120--123`) so their beam search and BLEURT judge never run, and executed their `main.py` otherwise untouched – the only edit being two device strings (`main.py:77--78`) so it runs on one GPU rather than three. All values at their fixed `proj_dim = 256`.

4. The controlled experiment

Everything is held fixed – their hidden states, their `lm_head` SVD, their projection, their MLP, their hyperparameters, the same five seeds – and **only the unit of the 75/25 split is varied**. Their

own `utils.split_data` is called for the answer-level arm, so that arm is their code path exactly; it doubles as a control and reproduces what their `main.py` printed.

Three things follow, and the first is the most important.

(i) This is a property of the protocol, not of their method. We are not immune. Run our own conditions under the answer-level split and they inflate too, by 1.3 to 11.8 points. Any evaluation that splits multi-sample generations over answers rather than questions inherits this, whoever is doing it. We report our own inflated numbers here for exactly that reason: the finding would be worth little if we exempted ourselves from it.

(ii) The mismatch is worth 3.8 to 13.3 points to HARP’s detector, mean 9.1. Their README states that running the given command obtains the paper’s experimental results, so the published numbers were produced under the answer-level split, not the protocol §3.1 describes. Our own conditions inflate less on all four datasets (6.31 vs 7.62, 1.33 vs 3.75, 11.75 vs 13.30, 6.42 vs 11.85), but this is confounded with the readout difference and we report it as an observation rather than a claim about mechanism.

(iii) Our unsupervised conditions beat their supervised detector on all four datasets under both protocols. The earlier version of this claim held only under the paper’s protocol; it now no longer depends on which protocol a reader accepts.

A memorisation signature, from the data rather than from argument. Under the question-level split, logistic regression beats the random forest on TriviaQA for *every one* of our six conditions. Under the answer-level split the ranking inverts – the forest wins there, and on every other dataset too (TriviaQA q_static: 94.03 RF vs 93.02 LR answer-level, against 91.7 RF vs 92.7 LR question-level). A forest can carve out per-question regions in a way a linear model structurally cannot, so the classifier better at memorising is precisely the one that gains when the split lets it memorise. The ranking flip is what one would predict if the extra points come from recognising the question rather than judging the answer.

5. A prediction we got wrong, and the corrected mechanism

An earlier draft of this document predicted the effect would scale with the share of test *answers* drawn from questions seen in training – large on TruthfulQA (52%), negligible on NQ-Open (3%). **The data refutes this.** NQ-Open has the smallest such share and the largest effect (−13.30).

The error was in the quantity. AUROC is built from (hallucinated, correct) pairs, and *every such pair requires a correct answer*. By construction a correct answer can only occur inside a known question – an unknown question is one where all ten answers were judged wrong. Unknown questions therefore contribute only one class and form no discriminative pairs at all. On NQ-Open the 32,500 unknown answers are 97% of the test set and contribute nothing to the metric; all ≈280 correct answers sit in the known group, where 93% of questions straddle the split. The leakage touches essentially *all* of the usable signal, on every dataset. That is why the effect is large everywhere.

What the effect size does track, cleanly and inversely, is the **number of distinct known questions**: TyDiQA-GP 302 → 11.85, NQ-Open 360 → 13.30, TruthfulQA 666 → 7.62, TriviaQA 6,316 → 3.76. With few questions to memorise, memorisation is cheap and there is little diversity to generalise from; with six thousand, the detector is pushed toward a transferable rule.

6. Caveats

- **TriviaQA is partial.** Our answer-level cell there covers 3 of 6 conditions (`core_max`, `q-velocity`, `q-static`). The remaining three are running; since the cell is a maximum, they can only raise it, so the −1.33 inflation figure is a lower bound.

- **Readouts differ.** Our conditions use a random forest or logistic regression on pooled multi-layer features; theirs is an MLP over per-token projections with a max aggregation. That is the method difference under test, but it is not an ablation of the split alone across both systems.
- **NQ-Open is noisy.** Its question-level arm carries ± 2.24 across seeds, because only ≈ 280 of 33,400 validation answers are correct. Treat its individual value loosely; the direction is unambiguous.
- **TriviaQA labels differ slightly.** Their `DatasetInit.py:103` folds `normalized_aliases` into the correct-answer list and ours does not, so their published 92.8 was computed over a marginally different known/unknown partition. The near-exact reproduction on that dataset (+0.09) suggests the difference is cosmetic, but it is unmeasured.
- **Their LLaMA is the base model** (`main.py:20`, `meta-llama/Llama-3.1-8B`), ours is `Instruct`. Paper and code agree with each other, so this is our deviation, not theirs – which is why every number here is Qwen, where the models match exactly.

This reads as a mismatch between the write-up and the released implementation rather than anything deliberate: the paper’s formalism is precise, and the method itself is implemented faithfully. The consequence for the field is the same either way.

Item	What the paper says	What the code does	Match?
Known / unknown grouping	§3.1: a question is “known” if any of its generated answers matches the reference. Appendix A: the unknown questions all go to the test set.	Correct. <code>get_known.py:115</code> marks a question known if any of its 10 answers is judged correct; <code>main.py:264</code> puts <i>every</i> unknown question into the test set and none into training; the 75/25 ratio is applied only to the known group.	Yes
How the known group is split	§3.1: “Let $\mathcal{X}_{known} = \{x \mid known(x) = 1\}$ denote the set of all inputs whose knowledge is known to the LLMs.” Appendix A: “75% of $\mathcal{X}_{known}$ is used for training, while the remaining 25%, together with $\mathcal{X}_{unknown}$, is used to test the hallucination detector on unseen data .”	Within the known group, splits individual answers , not questions. <code>main.py:184--201</code> builds one entry per answer; <code>get_hidden_state.py:72--79</code> stores one vector per answer; <code>main.py:237--246</code> zips these into one list; <code>main.py:259</code> splits that list 75/25 with a <code>torch.randperm</code> shuffle (<code>utils.py:49--58</code>).	No
Answers per question	Appendix A: “beam search is used to generate 10 answer paths per question.”	10 requested (<code>main.py:157--158</code>), but answers decoding to empty text are dropped (<code>get_known.py:93--94</code>), so a question can end up with fewer.	Mostly
Detector architecture	Appendix A: “a two-layer MLP with hidden dimension 1024 and ReLU activation.”	<code>mlp_trainer.py:27--31</code> : <code>Linear(input,1024) → ReLU → Linear(1024,1) → Sigmoid</code> .	Yes
Score for a QA pair	Eq. 16: “The maximum score among all tokens is taken as the hallucination score of the QA pair.”	<code>mlp_trainer.py:56, 192, 213</code> : <code>masked_scores.max(dim=-1)</code> , padding masked out.	Yes
Training setup	Appendix A: 50 epochs, Adam, lr 1e-4, cosine decay, batch size 128, weight decay 3e-4.	<code>mlp_trainer.py:282</code> Adam(lr, weight_decay); <code>mlp_trainer.py:12</code> CosineAnnealingLR; values from <code>main.py</code> ; batch size from the README command.	Yes
Correctness label	§5.1: correct if ROUGE-L > 0.7 or semantic similarity > 0.5.	<code>DatasetJudge.py:106--112</code> : same thresholds, plus an unmentioned margin condition whose default of -1 (<code>DatasetJudge.py:26</code>) makes it almost always true.	Yes (see note)
Subspace dimension	§5.3: “a dimension of 256 yields the best performance.” Fixed globally; every Table 1 value is reported at it.	<code>main.py:59</code> sweeps $\{32, \dots, 1024\}$ and prints all; <code>Reasoning_Patch.ipynb:263</code> hardcodes 256.	Yes

Qwen2.5-7B	TruthfulQA	TriviaQA	NQ-Open	TyDiQA-GP	mean $ \Delta $
HARP published (their data)	88.1	92.8	84.0	88.4	–
Their code, our data	86.27	92.89	83.12	91.87	–
Δ	−1.83	+0.09	−0.88	+3.47	1.57

Table 1: Mean signed deviation +0.21. Their method reproduces on our data with no systematic bias in either direction, which is what licenses the controlled experiment below.

Method	Split protocol	TruthfulQA	TriviaQA	NQ-Open	TyDiQA-GP
HARP	answer-level (<i>their code</i>)	85.16 $\pm$ 1.23	92.50 $\pm$ 0.24	81.03 $\pm$ 2.41	91.75 $\pm$ 0.95
HARP	question-level (<i>their paper</i>)	77.54 $\pm$ 1.51	88.75 $\pm$ 0.33	67.73 $\pm$ 2.24	79.90 $\pm$ 2.87
	<i>cost of the mismatch</i>	−7.62	−3.75	−13.30	−11.85
Ours	answer-level (<i>their code</i>)	94.11	94.03*	91.15	94.72
Ours	question-level (<i>their paper</i>)	87.80	92.70	79.40	88.30
	<i>cost of the mismatch</i>	−6.31	−1.33	−11.75	−6.42
our margin over HARP , answer-level		+8.95	+1.53	+10.12	+2.97
our margin over HARP , question-level		+10.26	+3.95	+11.67	+8.40
<i>known questions</i>		666	6,316	360	302

Table 2: Pooled AUROC (%) on Qwen2.5-7B, 5 seeds; HARP at its fixed proj_dim = 256. Within each method the two rows differ in *nothing* but the split unit. Our rows are the best of six unsupervised feature conditions under an RF/LR readout, chosen the same way on both rows – best of six on one and a pre-selected single on the other would bias the comparison. *TriviaQA answer-level currently covers 3 of 6 conditions; the remaining three are running and can only raise it.